



THE EDUCATOR · CPCS 351

Topic 03

System Engineering

A complete, study-ready edition of the live Topic 03 content -
with the exact repository image assets and the approved Golden Master design.

Whole-System View	System Process	Requirements
Architecture	Allocation	Integration & SCM

8 lessons · complete lecture content · activities · self-feedback · original visual anchors

Golden Master · v1.0

Contents

Eight lessons building one system-engineering story: define the whole system, follow the engineering process, derive whole-system requirements, design the architecture, allocate responsibilities, choose the right architectural views, integrate and deploy, then control system evolution through configuration management.

System Engineering	3
System Engineering Process	8
System Requirements	12
System Architectural Design	15
Requirements Allocation	18
Architectural Design Diagrams	22
Integration, Deployment, and Maintenance	29
System Configuration Management	32

Golden Master rule

This PDF preserves the live Topic 03 instructional content and uses the exact image files referenced by the HTML pages. No repository image is replaced by a generated or alternative visual.

System Engineering

1

of 8 lessons

System engineering is the multidisciplinary discipline that develops the whole system - not only its software - across the full system life cycle.

Learning Focus

Define a system, explain its main characteristics, distinguish system engineering from software engineering, and recognize why multidisciplinary coordination is necessary.

PRACTICE LAB · TOPIC 03

Practice Lab - Topic 03

Apply the Topic 03 story as one continuous system-engineering mission: define the whole system, move through the engineering process, distinguish requirements from architecture and allocation, choose the right architectural view, reason about integration and deployment, and control system evolution.

CHAPTER MAP · ADDED STUDY MATERIAL

The complete engineering story at a glance

This chapter follows one continuous chain. Keep this map in mind as you move through the lessons:

1	Business Goal
2	Current Situation
3	Business Need
4	System Requirement
5	Architecture
6	Allocated Responsibility
7	Subsystem Development
8	Integration & System Testing
9	Deployment
10	Controlled Evolution

Reading rule: needs describe the problem worth solving; requirements describe what the whole system owes stakeholders; architecture defines the parts; allocation defines what each part is accountable for.

CORE DEFINITION

What is a system?

A system is a set of interrelated components. It may be large or small, simple or complex, natural or man-made, physical or conceptual. What makes it a system is not its size - it is the fact that its parts are related and work together as a whole.

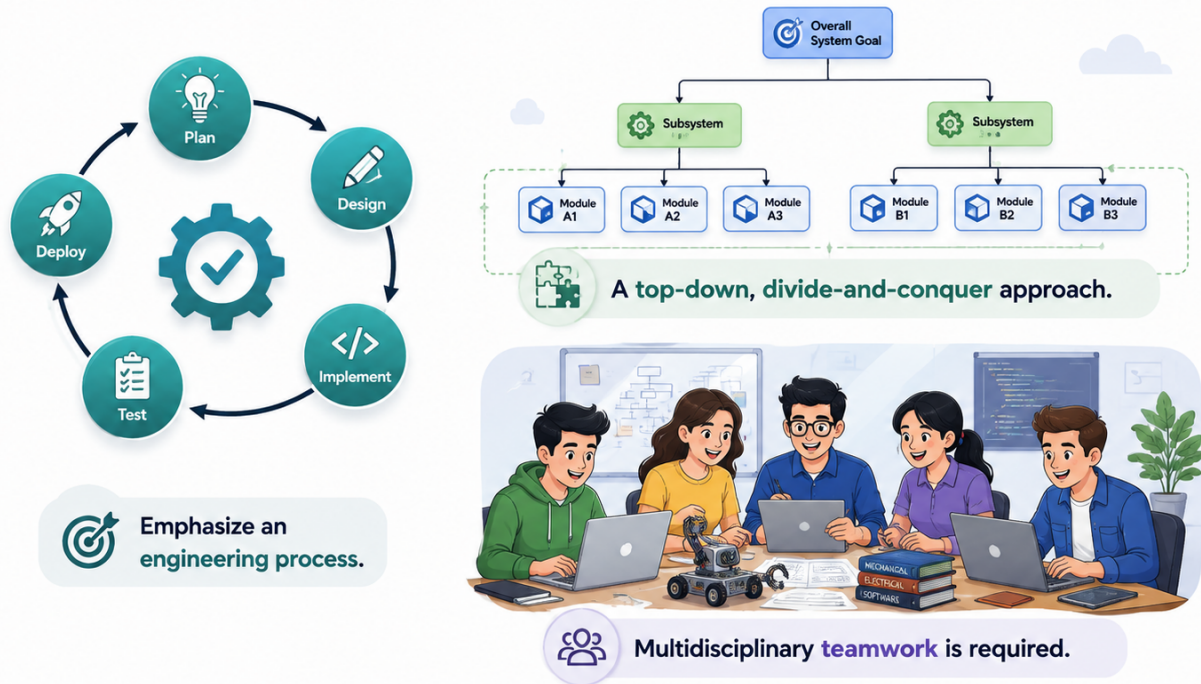
Natural The universe or an ant: systems whose interacting parts exist naturally.	Conceptual Mathematical logic or a measurement system: a system may exist conceptually rather than physically.	Computer-based A man-made system that includes software as one subsystem.
Software-only An email system can be treated as a software-only system at an appropriate boundary.	Embedded / multidisciplinary ABHS combines hardware, software and human components that interact to accomplish a mission.	

MAIN CHARACTERISTICS OF A SYSTEM

Four ideas you should be able to recognize

1 · Interacting components A system is not merely a collection. Components exchange information, material, control, energy or responsibilities.	2 · Hierarchy of systems A system may contain subsystems and may itself be a subsystem of a larger system.
3 · Environment & context Every system exists in an environment and interacts with external people, devices, organizations and systems.	4 · Evolution Systems change over time as needs, technology, regulations, versions and operating conditions change.

Characteristics of System Engineering



Visual anchor: system engineering coordinates multiple disciplines and system elements as one whole.

WHAT IS SYSTEM ENGINEERING?

Three defining characteristics

Life-cycle engineering process

The process spans requirements, architecture, development, integration, deployment, maintenance and change.

Top-down divide-and-conquer

Engineers start with the whole-system mission, decompose it into subsystems and components, then later integrate the developed parts.

Interdisciplinary development

Hardware, software and human/operational teams must coordinate because many system problems cross discipline boundaries.

CORE RELATIONSHIP

Software engineering is part of system engineering

System engineering defines the system requirements and constraints, allocates requirements to hardware, software and human subsystems, coordinates their development, and integrates them to form the complete system. Software engineering develops the software subsystem within that broader system context.

Whole system: HW + SW + humans + procedures + environment

Software subsystem: requirements + design + implementation + testing + evolution

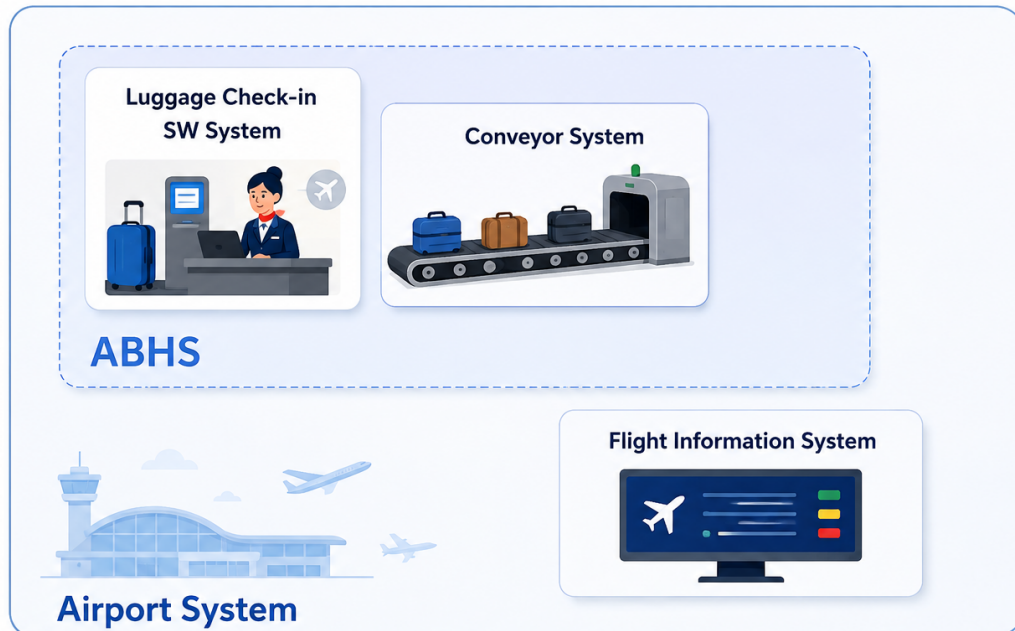
OFFICIAL RUNNING EXAMPLE



Airport Baggage Handling System (ABHS)

Example (ABHS)

- Airport Baggage Handling System.



Read from the inside out: components form subsystems, subsystems form ABHS, and ABHS operates inside the larger Airport System.

Subsystem level:

System level:

Larger-system level:

Why SE matters:

TRANSFER EXAMPLE · ADDED STUDY MATERIAL

Smart Hospital Medication Delivery System

Imagine a hospital system that receives medication orders, verifies patient identity, dispenses doses, transports medication through delivery robots, alerts nurses, and records administration. It includes software (orders, routing, alerts), hardware (dispensers, robots, scanners), and humans/procedures (pharmacists, nurses, safety checks). The point is not the hospital domain itself - the point is transfer: if you can identify the system boundary, subsystems, environment and cross-discipline interactions here, you understand the concept rather than memorizing ABHS.

Apply it · Is this a system-engineering problem?

Scenario: A hospital delivery robot reaches the correct room, but the nurse cannot administer the medication because the patient-ID scanner uses an incompatible identifier format.

Reveal reasoning: Yes. The failure is at the interface among hardware, software and operational workflow. Fixing only the robot software may not solve the whole-system problem.

Misconception: “System engineering is software engineering at a larger scale.”

No. The boundary is broader. System engineering coordinates multiple engineering disciplines and human/organizational elements across the entire system life cycle.

Key takeaway

A system succeeds only when its interrelated parts, interfaces and environment work together to accomplish the mission.

System Engineering Process

2

of 8 lessons

The process moves from whole-system needs to subsystem development and then back to an integrated, deployed and maintained whole.

Learning Focus

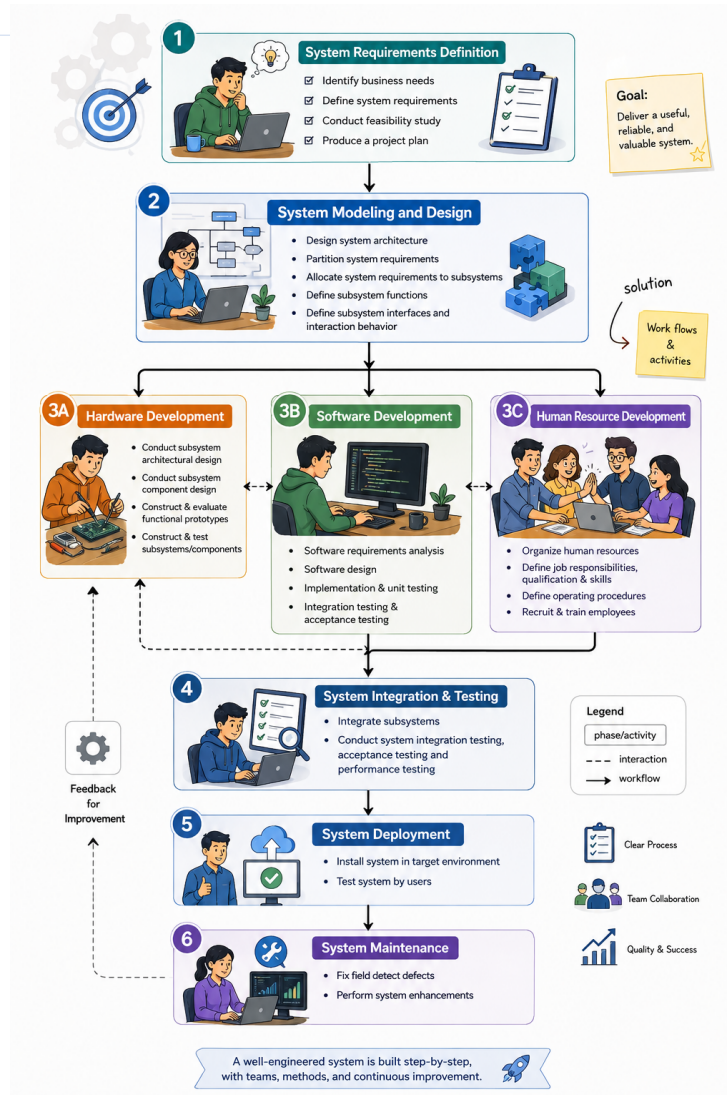
Order the major system-engineering activities, explain the top-down and bottom-up movements, and show where hardware, software and human-resource development interact.

PROCESS LOGIC

Top down to define; bottom up to integrate

System engineering follows a top-down divide-and-conquer approach during definition and design: begin with the system mission, derive requirements, model the system, decompose it and allocate responsibilities. After the subsystems are developed, the direction reverses: engineers integrate and test the parts until the complete system is ready for deployment.

1	Business goals & current situation
2	System requirements
3	System modeling & design
4	Subsystem development
5	Integration & testing
6	Deployment
7	Maintenance



The official process is not a software-only workflow: hardware, software and human-resource development interact before system integration and testing.

INTERDISCIPLINARY DEVELOPMENT

Subsystem work happens in parallel - but not independently

Hardware development	Software development	Human-resource development
Devices, sensors, conveyors, processors, tracks and other physical components are engineered to satisfy allocated requirements and interfaces.	Control, data processing, interfaces, monitoring and other software capabilities are developed under the same system constraints.	Roles, staffing, training, procedures and operational responsibilities are developed because people are part of the system.

These teams must collaborate on interdisciplinary problems. A hardware change can alter software timing; a software interface can alter operator procedure; a staffing assumption can affect throughput.

STAGE BY STAGE

What each phase contributes

1 · System Requirements Definition Understand business goals and the current situation, derive needs, select the feasible subset, and express required system capabilities and constraints.	2 · System Modeling & Design Decompose the system, choose subsystem boundaries, define interfaces, visualize the architecture and allocate requirements.
3 · Subsystem Development Different engineering teams develop hardware, software and human/operational subsystems under agreed requirements.	4 · System Integration & Testing Assemble subsystems, check interoperability and verify that the complete system satisfies system requirements and constraints.
5 · Deployment Install the integrated system in the target environment and test it under real operational conditions.	6 · Maintenance Manage changes and evolution while keeping the full system configuration consistent.

WORKED EXAMPLE**Trace one ABHS need through the process**

Need: move checked baggage to the correct destination reliably and on time. Requirements: define transport, throughput, locating and security obligations. Design: choose check-in, conveyor, scanner, control-software and human subsystems. Allocation: assign detailed responsibilities and quantitative targets. Development: teams build their parts. Integration: test baggage end-to-end. Deployment: install in the airport. Maintenance: coordinate later hardware, software and procedural changes.

TRANSFER EXAMPLE · SMART HOSPITAL**The same process in a different domain**

Goal: reduce medication-delivery delays. Need: deliver the correct dose to the correct patient reliably. Requirements: patient identification, dose verification, delivery time, availability and safety constraints. Design: pharmacy software, dispensing hardware, delivery robots, nurse workflow and interfaces. Development: each discipline builds its subsystem. Integration: verify robot, scanner, software and workflow interoperability. Deployment: validate the system on real wards. Maintenance: coordinate software, robot firmware and clinical-procedure changes.

Apply it · Where does this problem belong?

Scenario: The robot navigation subsystem works, the medication-order software works, but the hospital cannot release the system because nurses have not been trained on the emergency-stop procedure.

Reveal reasoning: This is not a software-only defect. Human-resource development is part of the system-engineering process; the system is not deployable until the human subsystem is ready.

Misconception: “Once requirements are written, each discipline can work independently until integration.”

No. The lecture explicitly emphasizes interaction among development activities. Interfaces, shared constraints and interdisciplinary issues require continuous coordination.

Key takeaway

System engineering decomposes the whole into developable parts, coordinates parallel engineering, then integrates the parts back into a tested whole.

System Requirements

3

of 8 lessons

System requirements define the capabilities and constraints of the complete system before responsibility is allocated to individual subsystems.

Learning Focus

Explain how business goals and the current situation lead to business needs, identify the information engineers must collect, and interpret the official ABHS system requirements.

FROM BUSINESS GAP TO SYSTEM REQUIREMENT

Requirements begin with the business - not the solution

1	Business goals
2	Current situation
3	Discrepancies
4	Business needs
5	Feasible subset
6	System requirements

Business needs are derived from the gap between the desired goals and the current situation. Because budget and schedule are limited, the system normally satisfies only a selected feasible subset of all identified needs. The selected needs are then expressed as system capabilities and constraints.

INFORMATION COLLECTION TECHNIQUES

Use multiple sources to understand the real system

Customer presentation	Study current business operation	User survey
User interview	Literature survey	

No single technique gives the whole picture. Presentations reveal goals, observation reveals what actually happens, surveys provide breadth, interviews provide depth, and documents/literature reveal rules, standards and prior knowledge.

FOCUS OF INFORMATION COLLECTION

Nine questions engineers should be able to answer

1 · Business What business will the system automate or support?	2 · Environment What is the system context and what external systems, people and constraints surround it?	3 · Goals What are the business goals or product goals?
4 · Current situation How does the current business operate today?	5 · Existing processes What processes exist and how do they relate to each other?	6 · Problems What problems, bottlenecks, failures or inefficiencies exist in the current system?
7 · Users Who uses the current system and who will use the future system?	8 · Wants & priorities What do customers and users want, and which needs matter most?	9 · Quality, performance & security What nonfunctional considerations must the future system satisfy?

OFFICIAL ABHS SYSTEM REQUIREMENTS

These are whole-system obligations

ID	Official requirement	What kind of obligation?
R1	ABHS shall check in and transport luggage to departure gates and baggage claim areas according to passenger destinations.	End-to-end functional capability.
R2	ABHS shall allow airline agents to inquire about luggage status and to locate luggage.	Information/query capability.
R3	ABHS shall check all baggage and detect items that are prohibited.	Security/screening capability.
R4	ABHS shall be able to serve 20,000 passengers per day.	Quantitative performance/capacity requirement.

Notice that none of these requirements says which software class, hardware device or human role implements it. That decision comes later during architecture and allocation.

ADDED CLARIFICATION

Turn vague needs into testable statements

"The baggage system should handle busy periods."	State the measurable capacity, accuracy, availability or response target the complete system must satisfy.
--	--

TRANSFER EXAMPLE · SMART HOSPITAL

From a hospital problem to a system requirement

Goal: reduce medication delivery delays and errors. Current situation: nurses frequently wait for pharmacy delivery and occasionally receive medication for the wrong ward. Need: faster, traceable, patient-safe medication delivery. A system requirement might state: “The system shall verify patient identity before medication release and record every delivery event.” A performance requirement could define a measurable delivery-time target. The exact robot, scanner or software module responsible is deliberately not chosen yet.

Apply it · Need, requirement, or allocation?

Statement: “Barcode scanner B shall read a medication label within 300 ms.”

Reveal reasoning: This is already a lower-level allocated/component requirement because it names a specific component and a measurable responsibility. A system requirement would state what the whole medication-delivery system must achieve without prematurely choosing the responsible part.

Misconception: “A system requirement should already identify the module or device that implements it.”

Not necessarily. System requirements describe what the whole system owes its stakeholders. Allocation later assigns that responsibility to subsystems and components.

Key takeaway

Understand the business and system context first; define whole-system capabilities and constraints second; allocate responsibility only afterward.

System Architectural Design

4

of 8 lessons

Architecture decomposes the system into a hierarchy of cohesive, loosely coupled subsystems whose responsibilities and interfaces can be engineered separately and later integrated.

Learning Focus

Explain why systems are decomposed, apply the six decomposition guidelines from the lecture, and distinguish functional partitioning from HW/SW/Human partitioning.

CORE LECTURE

What architectural design must accomplish

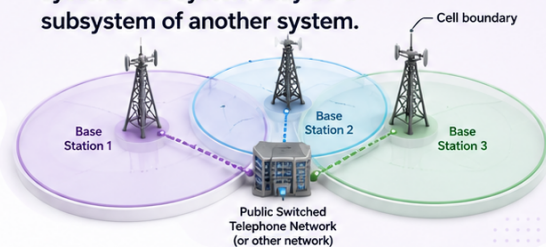
System architectural design has three connected jobs: decompose the system, allocate system requirements, and visualize the resulting architecture. The decomposition should produce functionally cohesive, loosely coupled subsystems, support reuse of COTS components, and make system responsibilities explicit.

Main Characteristics of a System

- 1 A system consists of interacting components.



- 2 A system exists in a hierarchy of systems – a system may be a subsystem of another system.



- 3 Each system exists in an environment and interacts with the environment.



- 4 Systems are ever evolving.



Systems are made of **parts**, part of **larger wholes**, interact with their **environment**, and continuously **evolve**.



Components



Hierarchy



Environment



Evolution

Visual anchor: architecture turns one system boundary into a coordinated hierarchy of subsystems and components.

ARCHITECTURE VS. ALLOCATION

Do not confuse the boxes with their obligations

Architecture

Decides what major parts/subsystems exist, where boundaries are drawn, and how those parts relate through interfaces.

Requirements Allocation

Decides what each part is responsible for satisfying, and refines whole-system requirements into accountable lower-level obligations.

Memory rule: Architecture answers “What parts do we have?” Allocation answers “What must each part do?”

SIX DECOMPOSITION GUIDELINES**A good partition is more than a neat drawing****1 · Separate engineering teams**

The result should allow different teams to develop subsystems with manageable responsibilities.

2 · Facilitate COTS use

Boundaries should make it practical to adopt commercial off-the-shelf components where suitable.

3 · Partition requirements

Subsystems should partition - or nearly partition - the system requirements so ownership is clear.

4 · Well-defined functionality

Each subsystem needs a coherent purpose rather than an arbitrary collection of tasks.

5 · Relative independence

Subsystems should be loosely coupled enough that teams can reason about and change them without constant ripple effects.

6 · Easy integration

Interfaces and responsibilities should support later assembly and interoperability testing.

TWO OFFICIAL PARTITIONING STRATEGIES**Different boundaries expose different engineering views****Partition according to major functionality**

Group components that cooperate to deliver a major system function. Each functional subsystem may itself contain HW, SW and human elements.

Check-in subsystem → HW + SW + human
Transport subsystem → HW + SW + human
Security/scanning subsystem → HW + SW + human

Use when: you want each subsystem to own an end-to-end business or operational function.

Partition according to HW, SW & Human subsystems

Group the system first by engineering discipline, then decompose within each discipline.

Hardware subsystem → devices, conveyors, scanners
Software subsystem → control, data, interfaces
Human subsystem → operators, procedures, staffing

Use when: discipline-specific engineering ownership and specialization dominate the organization of work.

WORKED EXAMPLE

ABHS architecture

A functional view can decompose ABHS into check-in, transport, scanning, routing/monitoring and human operations. A discipline-oriented view can instead emphasize hardware, software and human subsystems. Neither is universally “correct”; the useful architecture is the one that produces coherent responsibilities, manageable interfaces, requirement coverage and feasible integration.

TRANSFER EXAMPLE · SMART HOSPITAL

Two valid architectural views of the same hospital system

Functional partition: Prescription Validation, Dispensing, Delivery, Patient Verification, and Clinical Monitoring - each may contain hardware, software and humans. Discipline partition: Hardware, Software and Human/Clinical Procedure subsystems. The choice changes team boundaries and interfaces, but the system mission remains the same.

Apply it · Which partitioning strategy is this?

Scenario: A hospital architecture has three top-level subsystems named “Medication Ordering,” “Medication Delivery,” and “Patient Verification.” Each contains software, devices and human roles.

Reveal reasoning: This is partitioning according to major functionality because each subsystem owns an end-to-end operational capability rather than one engineering discipline.

ARCHITECTURE → ALLOCATION

Why the next lesson matters

Architecture identifies the parts and interfaces. Requirements allocation then makes those parts accountable by assigning or refining whole-system obligations into subsystem and component requirements. A subsystem without allocated responsibility is only a box; allocation turns it into an engineering commitment.

Misconception: “A good architecture minimizes the number of subsystems.”

The objective is not the fewest boxes. The objective is meaningful cohesion, manageable coupling, clear responsibility, feasible integration and complete requirement coverage.

Key takeaway

Good decomposition creates subsystem boundaries that teams can build independently enough to work efficiently - but integrate reliably enough to satisfy the whole-system mission.

Requirements Allocation

5

of 8 lessons

Requirements allocation turns whole-system obligations into clear, measurable responsibilities for subsystems, components, software, hardware and human procedures.

Learning focus

Explain progressive requirement refinement, decompose ABHS requirements across useful levels of detail, and trace lower-level requirements back to the original system requirement.

WHAT IS REQUIREMENTS ALLOCATION?

From “what the system must do” to “what each part must do”

Allocation distributes or refines a high-level requirement into more specific obligations assigned to the subsystem, component, interface, software, hardware or human element that will satisfy it.

1 · System intent What must the whole system achieve? 2 · Allocate Which part owns the responsibility? 3 · Refine What exactly must that part achieve? 4 · Verify Can we measure and test it?

Level 1 System Whole-system obligation: what the complete system must achieve.

Level 2 Subsystem Responsibility assigned to a major subsystem or organizational part.

Level 3 Component / Mechanism Responsibility refined into specific hardware, software, interface or human mechanisms.

Level 4 Detailed / Design Specific, measurable and testable constraints used to guide implementation and verification.

OFFICIAL LECTURE EXAMPLE · ABHS

One hierarchy, increasing specificity

System Requirements

R1 ABHS shall check in and transport luggage to departure gates and baggage claim areas according to passenger destinations.

R2 ABHS shall allow airline agents to inquire about luggage status and locate luggage.

R3 ABHS shall check all baggage and detect prohibited items.

R4 ABHS shall serve 20,000 passengers per day.

Subsystem-Level Refinement

R1.1 ABHS shall allow airline agents to check in luggage.

R1.2 ABHS shall transport luggage to destinations within the airport.

R1.3 ABHS shall control transportation of luggage within and between terminals.

R4.x R4 must be distributed into capacity and performance targets across responsible subsystems.

Component/Mechanism Refinement

R1.2.1 Transport luggage from check-in areas to departure gates.

R1.2.2 Transport luggage from arrival gates to baggage claim areas.

R1.2.3 Transport luggage from arrival gates to departure gates for transfer passengers.

R1.2.4 Transport luggage within a terminal using conveyors.

R1.2.5 Transport luggage between terminals using DCVs running on high-speed tracks.

Detailed Quantitative Requirements

R4.1 Each check-in area shall handle 1,150 pieces of check-in luggage per day.

R4.2 Each check-in agent shall check in an average of three passengers per minute.

R4.3 Each conveyor hardware shall scan and transport 500 check-in pieces of luggage per hour.

R4.4 ABHS control software shall process 2,300 check-in bags per day and 1,000 bar code scan requests per hour.

TWO ALLOCATION PATTERNS IN THE SAME EXAMPLE**Functional decomposition vs. performance allocation****R1 · Functional refinement**

R1 begins as an end-to-end transport obligation. It is refined into R1.1, R1.2 and R1.3, then R1.2 is further decomposed into specific transport paths and mechanisms R1.2.1 - R1.2.5.

R4 · Quantitative allocation

R4 begins as a whole-system capacity target: 20,000 passengers/day. That target is distributed into measurable capacities for check-in areas, agents, conveyor hardware and control software.

SEQUENTIAL REFINEMENT · TOP DOWN**How the requirement travels through the design**

1	1 · High-level requirement Define the whole-system obligation.
2	2 · Allocate Assign responsibility to one or more subsystems.
3	3 · Refine Split broad obligations into detailed ones.
4	4 · Quantify Add measurable capacity, timing or quality targets.
5	5 · Trace & test Ensure every lower-level requirement supports a parent requirement.

Key idea

Lower-level requirements are not independent inventions. Together they must fully support the intent of their parent requirement.

TRACEABILITY ACROSS LEVELS**Keep the parent-child chain visible**

System	Subsystem	Component / mechanism	Detailed
R1 · Check in & transport luggage	R1.1 · Check-in R1.2 · Transport R1.3 · Control transportation	R1.2.1 - R1.2.5 · Specific routes/mechanisms	Additional implementation/testable details as design proceeds
R2 · Locate luggage	Information-access responsibilities	Data capture, query and display components	Measurable response/accuracy constraints when specified
R3 · Detect prohibited items	Security/screening responsibilities	Scanning hardware + detection/control software	Measurable detection/performance constraints when specified
R4 · 20,000 passengers/day	Capacity targets distributed across responsible subsystems	Check-in, conveyor hardware and control software	R4.1, R4.2, R4.3, R4.4

The lecture explicitly provides R1.x and R4.x examples. The R2/R3 rows above are structural clarification showing how traceability would continue; they do not introduce new official numeric targets.

TRANSFER EXAMPLE · SMART HOSPITAL

Allocate a medication-safety requirement

System requirement: “The system shall verify patient identity before medication release.” A plausible allocation might assign identity capture to the scanner hardware, identity matching to verification software, and an override/escalation responsibility to the nurse procedure. The important point is traceability: every allocated responsibility must support the original patient-safety obligation.

Apply it · Is this allocation complete?

Scenario: The team allocates patient-ID scanning to hardware and matching to software, but nobody owns what happens when the scanner cannot read a wristband.

Reveal reasoning: No. The parent requirement is not fully covered because the failure/exception path has no accountable subsystem or human procedure. Allocation must cover the complete intent, not only the happy path.

Misconception: “Allocation means copying the same requirement into every subsystem specification.”

No. Allocation assigns or decomposes responsibility. Each lower-level requirement must contribute to satisfying its parent, while traceability keeps the chain visible.

Exam lens:

Be able to distinguish system, subsystem, component and detailed views as useful refinement levels; show how R1 is decomposed and how R4 is quantitatively allocated; and explain why the lower-level set must collectively satisfy the original requirement.

Key takeaway

Architecture defines the parts. Requirements allocation makes each part accountable, measurable and traceable to the system-level need.

Architectural Design Diagrams

6

of 8 lessons

Different diagrams answer different engineering questions. Learn what each view is for, what its notation emphasizes, and how to read the official examples.

Learning Focus

Distinguish block, UML component, SysML block-definition, SysML internal-block, and data/material-flow views, and choose the view that best communicates the required relationship.

BEFORE THE DIAGRAMS

One system can need several views

Block diagram

Best for a simple high-level picture of major system parts and connections.

UML component

Best for software/component structure, ports, interfaces and communication paths.

SysML BDD / IBD

Best for multidisciplinary system structure, composition, ports and internal flows.

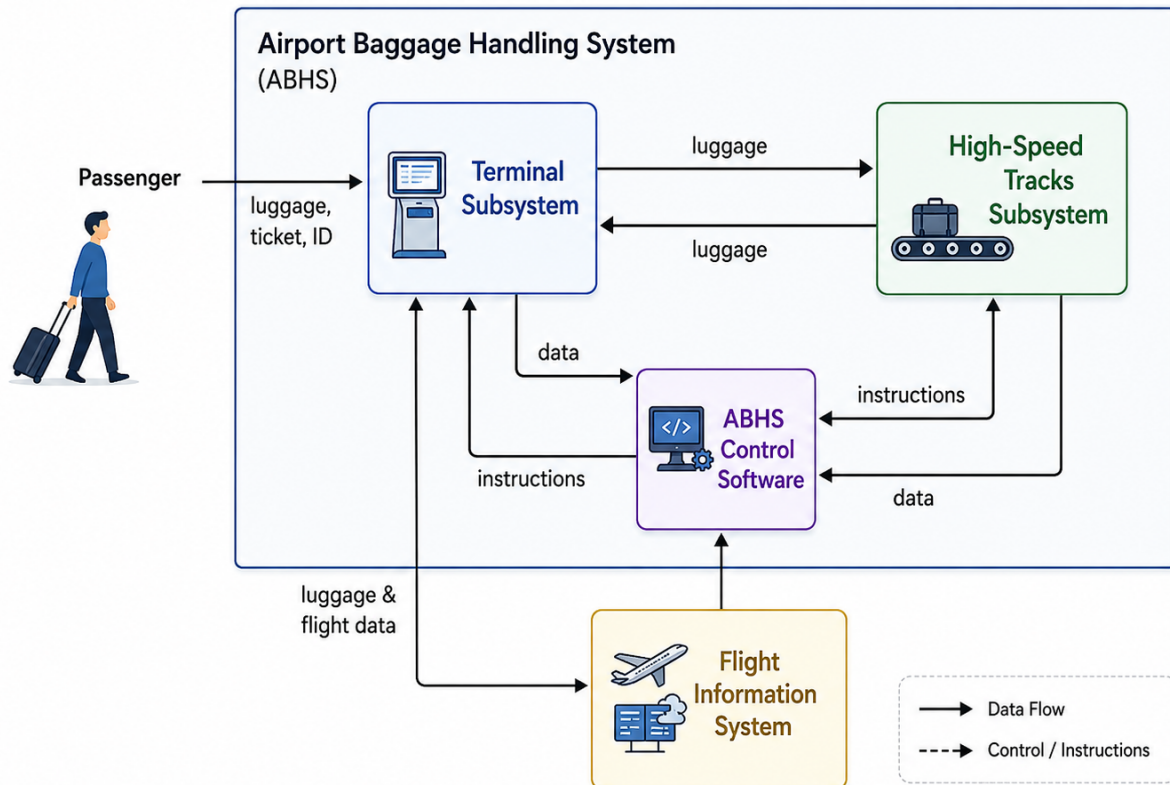
Data/material flow

Best for showing what information or physical material moves through functions and entities.

Use it for the big structural picture

A block diagram gives a fast, technology-neutral overview of the major subsystems and the flows or control relationships between them. It is ideal when the first question is simply: What are the major parts, and how do they interact?

Block Diagram



How to read it

Identify the major blocks first. Then follow arrows and labels such as luggage, data and instructions to understand the interactions.

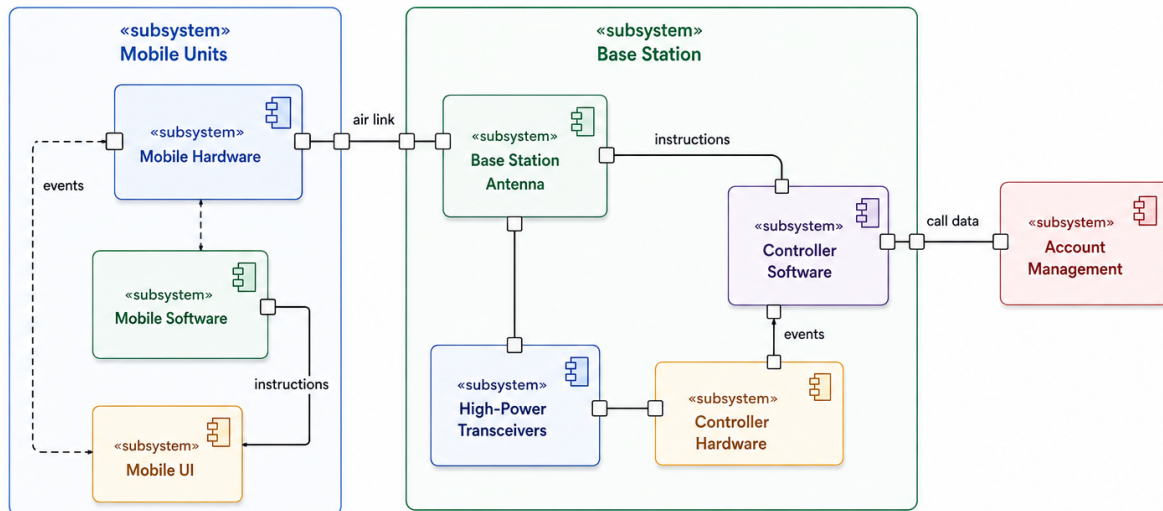
What this ABHS example reveals

The Terminal and High-Speed Tracks subsystems move luggage, while ABHS Control Software coordinates through data/instruction flows and interacts with Flight Information.

Use it when software/component interfaces matter

A UML component diagram emphasizes components, ports, interfaces, dependencies and communication paths. It is especially useful when software and hardware components are organized into subsystems and the engineering question is about who communicates with whom.

UML Component Diagram – Mobile Communication System



Legend:



component



port

«subsystem»

stereotype for introducing new modeling constructs

Notes:

- Ports define interaction points.
- Provided and required interfaces are implied by the ports.
- Connectors show communication paths between components.

How to read it

Start with subsystem boundaries, then identify components inside them. Small square ports mark interaction points; connectors show communication paths; labels such as events, instructions and call data tell you what travels across them.

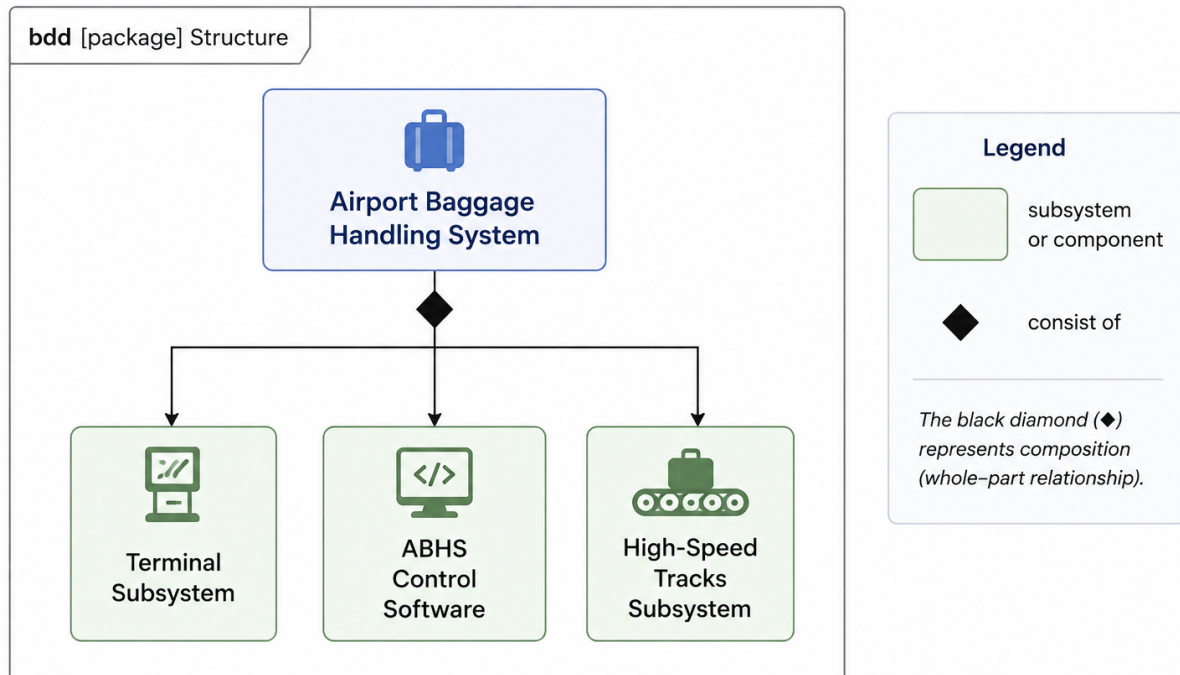
Why this view is useful

It makes interface responsibility explicit - valuable when separate teams must integrate independently developed components without ambiguity.

Use it to define the system's structural composition

A SysML BDD shows what blocks exist and how the whole is composed from parts. It is a strong fit for multidisciplinary systems because a block may represent hardware, software, people, facilities or other system elements.

SysML Block Definition Diagram



How to read it

The top block is the whole. The branches identify constituent blocks. In this example the black diamond represents composition - a strong whole - part relationship.

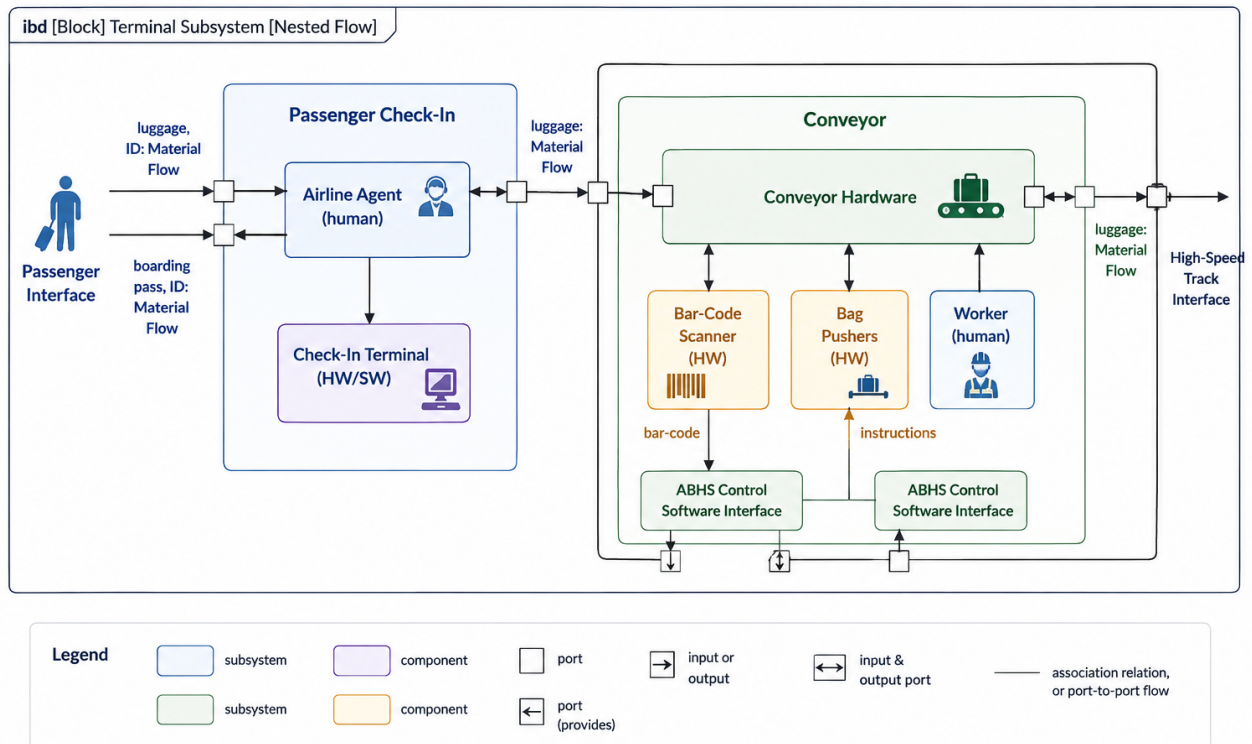
BDD vs. Block Diagram

Both show structure, but the BDD uses SysML semantics to express formal block relationships rather than only informal boxes and arrows.

Use it to see inside a block and follow its ports and flows

An IBD takes a block defined structurally and opens it up to show internal parts, ports and item/material/information flows. It answers a deeper question than the BDD: How do the parts inside this subsystem actually interact?

SysML Internal Block Diagram – Terminal Subsystem



How to read it

Follow the nested boundary first, then locate ports on block edges. Trace luggage, IDs, bar codes and instructions through the ports to see how human, hardware and software components collaborate.

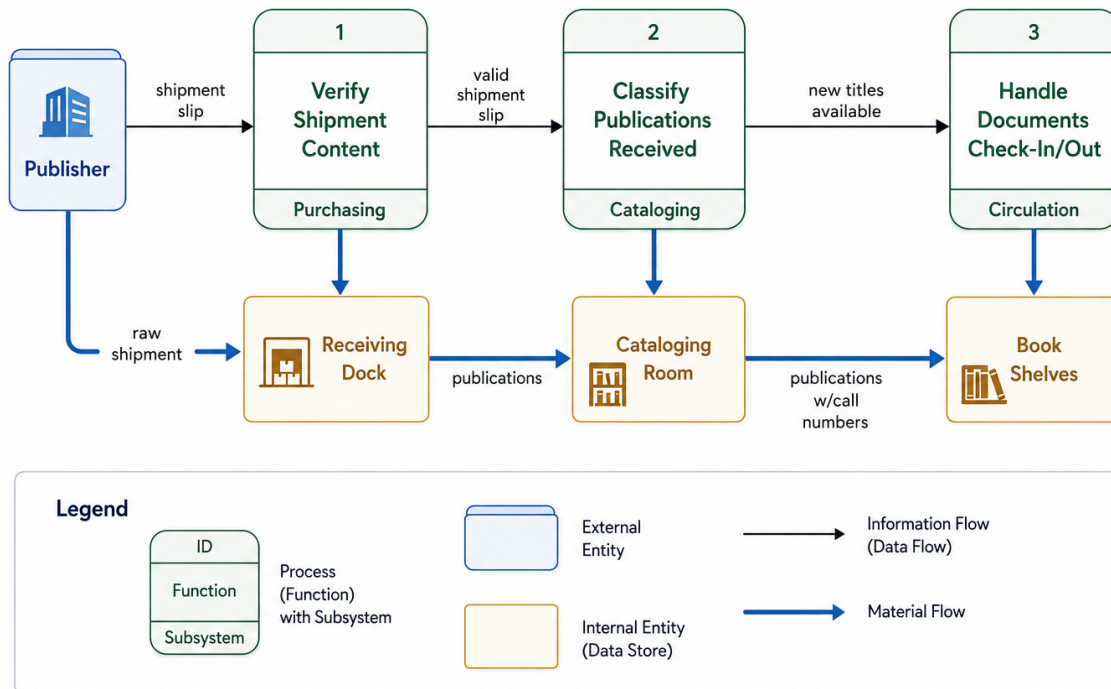
BDD vs. IBD

BDD: “What blocks make up the system?” IBD: “How do the parts inside one block connect and exchange flows?”

Use it when both information and physical items move through the process

A conventional data-flow view emphasizes information. In system engineering, the model can be extended to show material flow as well - useful when the system transforms or transports physical items such as luggage, publications, packages or products.

Data Flow Diagram with Material Flows



How to read it

Use the legend: processes/functions transform work, external entities sit outside the system boundary, internal entities/data stores hold material or information, and different arrow styles distinguish information flow from material flow.

Why this matters in SE

System behavior often includes both cyber and physical movement. A baggage system, for example, moves data about a bag and the bag itself - those are different flows that must remain synchronized.

STUDY COMPARISON

Which diagram should you choose?

Need a quick map of major subsystems? Block diagram. Need component interfaces? UML component diagram. Need formal multidisciplinary composition? SysML BDD. Need internal ports and nested flows? SysML IBD. Need to follow information and physical material through functions? Data/material-flow diagram.

Apply it · Choose the view

Scenario A: You must show hospital managers the major medication-delivery subsystems without overwhelming them with notation. Best choice: Block diagram.

Scenario B: Engineers must inspect how scanner hardware, verification software and nurse-workflow interfaces connect inside the Patient Verification subsystem. Best choice: SysML IBD.

Scenario C: You must distinguish the physical movement of medication from electronic prescription data. Best choice: Data/material-flow diagram.

Misconception: “The most detailed diagram is always the best diagram.”

No. A diagram is an abstraction chosen for a question. Extra detail can obscure the relationship you need to reason about.

Key takeaway

Do not ask “Which diagram is the correct one?” Ask “Which view makes this engineering relationship easiest to see?”

Integration, Deployment, and Maintenance

7

of 8 lessons

Subsystems developed by different engineering teams must interoperate as one system, satisfy the system requirements, work in the target environment, and remain consistent as they evolve.

Learning Focus

Explain why subsystem development requires collaboration, distinguish integration testing from system and deployment testing, and identify cross-subsystem problems that appear late if interfaces were poorly coordinated.

OTHER SYSTEM ENGINEERING ACTIVITIES

Development is distributed across engineering teams

The lecture emphasizes that different engineering teams develop different subsystems. These teams must collaborate to solve interdisciplinary problems because subsystem assumptions are connected through interfaces, shared constraints and the system mission.

Hardware ↔ Software A scanner may produce data at a rate or protocol the software did not expect.	System Integration Interfaces · assumptions constraints · shared mission	Software ↔ Human An alarm can be technically correct but operationally useless if the procedure does not tell staff how to respond.
Hardware ↔ Environment A conveyor may work in a lab yet fail under real airport loads, dust, temperature or physical layout.	Shared Performance End-to-end throughput depends on the slowest interacting subsystem, not the fastest individual component.	

TESTING SCOPE EXPANDS STEP BY STEP

From one part to the real operational system

1	1 Component / Subsystem Does this individual part satisfy its allocated responsibility?
2	2 Integration Do connected parts exchange information, timing and control correctly?
3	3 Complete System Does the integrated whole satisfy system requirements and constraints?
4	4 Target Environment Does the system still work under real operational conditions?

LOCAL PART	INTERFACES
------------	------------

WHOLE SYSTEM

REAL WORLD

Added clarification: the lecture explicitly emphasizes interoperability, system requirements/constraints, and testing in the target environment. The colored journey above makes the expanding scope visible: each stage keeps the previous concern and adds a wider system context.

THREE STAGES

Integrate → verify in the environment → maintain

1 · System Integration & Testing

Combine subsystems and components, test interoperability, and verify that the complete system satisfies system requirements and constraints.

2 · System Deployment

Install the system in the target operational environment and test it there - not only in the development environment.

3 · Maintenance & Evolution

Coordinate later changes across interdependent hardware, software and human procedures while preserving system integrity.

INTEGRATION IS WHERE ASSUMPTIONS MEET

What commonly breaks?

1	Interface formats
2	Timing
3	Protocols
4	Capacity
5	Physical constraints
6	Human procedures

Subsystems can each pass their local tests and still fail when connected. Integration testing therefore evaluates the relationships among parts, while system testing evaluates whether the complete integrated system satisfies the required behavior.

WORKED EXAMPLE

ABHS airport upgrade

Suppose a new scanner is introduced. The scanner hardware may work perfectly, but deployment can still fail if its firmware uses a new protocol, the control software expects the old message format, conveyor timing changes, monitoring dashboards interpret alarms incorrectly, or staff procedures are outdated. System engineering treats the change as a coordinated system change - not as a device replacement.

TRANSFER EXAMPLE · SMART HOSPITAL

Medication delivery robot release

Subsystem test: the robot navigates correctly and the scanner reads wristbands. Integration test: robot, scanner and medication-verification software exchange the right data. System test: the entire workflow satisfies patient-identification, delivery-time and safety requirements. Target-environment test: the system is exercised on real wards with elevators, Wi-Fi dead zones, clinical interruptions and actual nurse procedures.

Apply it · Which test level found the problem?

Scenario: Every subsystem passed locally, but when connected, the scanner sends patient IDs with leading zeros while the verification software removes them, causing false mismatches.

Reveal reasoning: This is an integration problem: each part may be locally correct, but their interface semantics are incompatible.

Misconception: “If every subsystem passes its own tests, the system is ready.”

No. Local correctness does not prove interoperability or whole-system compliance. The assembled system must be integrated, tested against system requirements and constraints, then tested in its target environment.

Key takeaway

Integration proves that the parts can work together; system testing proves the whole meets its obligations; deployment proves it can work in the real environment; maintenance keeps that whole consistent as it changes.

System Configuration Management

8

of 8 lessons

Configuration management keeps the evolving combination of hardware, software, procedures and other system components identifiable, controlled and mutually compatible.

Learning Focus

Explain why system configuration management is necessary, name its four functions, apply them to a multi-version system, and reconstruct the complete Topic 03 story.

WHY CONFIGURATION MANAGEMENT IS NEEDED

A system changes in more than one place at a time

The lecture gives two key reasons. First, a system may have different versions and releases for different customers. Second, multiple engineering teams may update system configuration concurrently. Without coordination, individually valid components can form an incompatible system.

Configuration management is therefore performed during both the development phase and the maintenance phase.

01 Software release v3.2	02 Scanner firmware v5	03 Conveyor controller rev C
04 Operator procedure rev B	05 Approved configuration baseline	

FOUR OFFICIAL SCM FUNCTIONS

Identify → control → audit → report

1 · Configuration Identification

Identify configuration items, versions, relationships and approved baselines so the team knows exactly what constitutes a system release.

2 · Configuration Change Control

Evaluate, approve, coordinate and record proposed changes so one team does not silently break another subsystem or release.

3 · Configuration Auditing

Check that the actual system configuration matches the approved specification and that required changes were implemented correctly.

4 · Configuration Status Reporting

Report the current state of configuration items, versions, approved changes and baselines so stakeholders know what is deployed or under development.

WORKED EXAMPLE**ABHS release compatibility**

A new ABHS software version expects a new scanner protocol. Configuration identification tells us which software and firmware versions belong together. Change control prevents deploying the software before the scanner update is approved. Auditing checks that the installed airport configuration matches the approved release. Status reporting tells operations which terminals have migrated and which still run the previous baseline.

TRANSFER EXAMPLE · SMART HOSPITAL**A safe medication-delivery release is a configuration, not a single app version**

Suppose release H2 requires medication software v4, robot firmware v7, scanner profile P3 and nurse procedure rev D. Identification defines that approved combination. Change control evaluates a proposed robot firmware update before it reaches wards. Auditing checks that deployed wards actually match H2. Status reporting shows which wards remain on H1 and which have migrated to H2.

Apply it · Which SCM function is missing?

Scenario: A change was approved and deployed, but nobody can tell which terminals received it or which baseline each terminal currently runs.

Reveal reasoning: Configuration status reporting is missing. Approval alone is not enough; stakeholders need the current state of configuration items and deployments.

TOPIC 03 · REBUILD THE STORY**From system boundary to controlled evolution**

1	1 Define the system. Recognize interacting components, system hierarchy, environment and evolution.
2	2 Apply the system-engineering process. Move top-down from goals and requirements to design, then bottom-up through integration, deployment and maintenance.
3	3 Define system requirements. Use multiple information sources to identify capabilities, constraints, priorities and quality/performance/security needs.
4	4 Design the architecture. Decide what parts exist, where boundaries are drawn and how parts relate through interfaces.

5	5 Allocate requirements. Decide what each part is accountable for and refine whole-system obligations while preserving traceability.
6	6 Visualize the architecture. Choose block, UML component, SysML or data/material-flow views according to the engineering question.
7	7 Integrate, test and deploy. Expand test scope from parts to interfaces to whole-system requirements and finally the target environment.
8	8 Control evolution. Use configuration management to keep versions and concurrent changes consistent.

WHOLE-CHAPTER TRANSFER CHALLENGE

Can you engineer the hospital system without copying ABHS?

For the Smart Hospital Medication Delivery System, identify: (1) the system boundary and environment, (2) one business need, (3) one whole-system requirement, (4) two candidate subsystems, (5) one allocated responsibility, (6) the best diagram for showing an internal scanner/software/nurse interaction, (7) one integration risk, and (8) one configuration item that must be version-controlled.

Reveal one defensible solution

Boundary: medication ordering through verified delivery to the patient; environment includes pharmacy systems, wards, Wi-Fi and clinical staff. Need: reduce delivery delay/error. Requirement: verify patient identity before medication release. Subsystems: dispensing and patient-verification. Allocation: scanner captures ID; verification software matches it; nurse procedure handles exceptions. Diagram: SysML IBD for internal ports/flows. Integration risk: identifier-format mismatch. Configuration item: scanner profile or robot firmware. Other answers can be correct if they preserve system logic and traceability.

CLASS DISCUSSION · OFFICIAL LECTURE QUESTIONS

Use these as final discussion prompts

Why is system engineering a multidisciplinary effort?	What is the relationship between system engineering and software engineering?	Give examples of systems that require a system-engineering approach, and justify why software engineering alone would be insufficient.
---	---	--

Misconception: “Configuration management is just source-code version control.”

No. System configuration management covers the coordinated versions and changes of all relevant system components - including hardware, software and operational artifacts - not only code.

Key takeaway

System engineering begins with the whole-system mission and continues through requirements, architecture, allocation, multidisciplinary development, integration, deployment and controlled evolution.

NEXT TOPIC

Topic 04

Process & Methodology

Carry forward the whole-system perspective: once the system boundary, requirements, architecture, interfaces, integration, and configuration are understood, the next question is how a software team organizes and governs the development process itself.

The Educator · CPCS 351 · Software Engineering I